

ORION

PART 02

ORION Architecture

Complete technical architecture and system structure

Technical Architecture Series • Beginner → Professional

How to use this volume

This document is one independent volume in the twelve-part ORION technical series. Beginners should read from the start; experienced readers can jump directly to the relevant architecture sections.

ORION is a secure communications, infrastructure-coordination, emergency-response, and decision-support architecture. High-impact weapons-control functionality is deliberately outside this design.

Architecture goals

Separate responsibilities into bounded services and infrastructure domains.

Make security, resilience, observability, and failure behavior explicit.

Professional lens: define ownership, interfaces, trust boundaries, failure behavior, security controls, observability, and measurable success criteria before implementation.

Reference architecture

Clients → API gateway → identity/policy → application services → event bus/workflows → AI/simulation → data stores → network adapters.

Professional lens: define ownership, interfaces, trust boundaries, failure behavior, security controls, observability, and measurable success criteria before implementation.

Core components

Identity Service; Policy Service; Communication Service; Incident Service; Telemetry Service; AI Orchestrator; Simulation Service; Audit Service; Network Adapter Service.

Professional lens: define ownership, interfaces, trust boundaries, failure behavior, security controls, observability, and measurable success criteria before implementation.

Boundaries

Public client, trusted service, restricted operator, AI execution, data, external integration, edge/field, and audit boundaries.

Professional lens: define ownership, interfaces, trust boundaries, failure behavior, security controls, observability, and measurable success criteria before implementation.

API architecture

Version APIs. Authenticate and authorize every request. Use correlation IDs, idempotency keys, rate limits, schemas, and explicit error contracts.

Professional lens: define ownership, interfaces, trust boundaries, failure behavior, security controls, observability, and measurable success criteria before implementation.

Event architecture

Events record facts; commands request actions. Use a schema registry or equivalent discipline, unique IDs, timestamps, correlation IDs, and consumer isolation.

Professional lens: define ownership, interfaces, trust boundaries, failure behavior, security controls, observability, and measurable success criteria before implementation.

Deployment architecture

Use multiple failure domains. Combine regional services with edge gateways. Keep critical dependencies from becoming single global failure points.

Professional lens: define ownership, interfaces, trust boundaries, failure behavior, security controls, observability, and measurable success criteria before implementation.

Trade-offs

Microservices improve isolation but increase operational cost. Central coordination simplifies policy but can create concentration risk. Edge improves resilience but complicates security and lifecycle management.

Professional lens: define ownership, interfaces, trust boundaries, failure behavior, security controls, observability, and measurable success criteria before implementation.

Architecture Review Checklist

- Requirements and assumptions are explicit.
- Component ownership and boundaries are documented.
- Normal, degraded, and failure states are defined.
- Security and privacy are architectural requirements.
- Capacity and scaling assumptions are measurable.
- Logs, metrics, traces, and audit events cover critical flows.
- Dependencies have explicit contracts and failure behavior.
- Important decisions record alternatives and trade-offs.
- Testing covers expected behavior and controlled failures.
- Operational runbooks exist for important incidents.

Key Takeaways

ORION Architecture should be implemented as a bounded layer of ORION, with explicit interfaces to the rest of the platform.

The goal is not maximum complexity. The goal is measurable resilience, security, maintainability, interoperability, and operational usefulness.

Glossary

- API — Application Programming Interface used for software-to-software communication.
- Edge — Computing placed near users or physical systems.
- Event — A record that something happened.
- Failover — Switching to a redundant component after failure.
- Idempotency — Repeating an operation does not create an unintended additional effect.
- NTN — Non-terrestrial network, commonly involving satellite infrastructure.
- Policy — A rule that determines what is allowed under stated conditions.
- RPO — Recovery Point Objective; acceptable data-loss window.
- RTO — Recovery Time Objective; target recovery time.
- Zero trust — Continuous verification instead of implicit trust based on network location.